Graph Database Design and Cypher Query

CITS5504 - Data Warehousing SEM-1 2025



23873626
Nisari Ramachandran

Contents

1 Introduction **2**

1.0.1 Dataset . . . 2

2 Design and Implementation **3**

2.1 Design . . . *3*

2.2 Rationale Behind the Design Choices . . . *4*

2.2.1 Schema choice . . . 5

2.2.2 Some design justifications . . . 5

3 Implementation **5**

4 Creating CSV files from Dataset provided **6**

4.1 Main dataset . . . *6*

4.2 Creating csv files for nodes . . . *6*

4.3 Creating csv files for relationships . . . *7*

4.3.1 Dealing with duplicates - Discussion . . . 7

5 Graph database Implementation **8**

5.1 Loading Nodes . . . *8*

5.2 Loading Relationships . . . *8*

6 Cypher Query **10**

7 Self Designed Queries **16**

8 Applications of Graph Data Science **19**

A Appendix **22**

1 Introduction

Road safety is a critical public concern in Australia, making the analysis of such incidents important for improving the road safety and making informed policy decisions.In this report, a graph database is developed to model and analyse the crash related data more effectively. Creating a graph database makes it easier to uncover patterns and run efficient queries on interconnected data, which provides deeper insights into the factors which contribute to the road fatalities.

This report outlines the design of property graph using the Arrows App ,A discussion about the design choices, the ETL process for transforming the data into Neo4j compatible CSVs, graph implementation in Neo4j using these CSVs. It also presents the given and self defined cypher queries and findings. Finally the report also discuss about the practical application of the Graph Data Science.

1.0.1 Dataset

For this project, a cleaned dataset[1] provided as a part of the project description by the University is used. The dataset is based on the Australian Road deaths data[2], but has been preprocessed by removing missing/unknown values, modifying the values and include additional columns for helping the analysis.

Table 1: Data dictionary describing the columns

Field	**Format**	**Comments**
ID	Integer	Surrogate key, unique identifier
Crash ID	Text	8 digits, not a unique identifier
State	Text	Australian jurisdiction
Month	Integer	Month of crash
Year	Integer	2014–2024
Dayweek	Text	Day of week of crash
Time	Time	hh:mm format
Crash Type	Text	Number of vehicles involved
Number Fatalities	Integer	Number of killed persons
Bus Involvement	Text	Indicates involvement of a bus
Heavy Rigid Truck Involvement	Text	Indicates involvement of a heavy rigid truck
Articulated Truck Involvement	Text	Indicates involvement of an articulated truck
Speed Limit	Text	Posted speed limit at crash location
Road User	Text	Road user type of killed person
Gender	Text	Sex of killed person
Age	Integer	Age of killed person (years)
National Remoteness Areas 2021	Text	ABS remoteness structure
Statistical Areas Level 4 (SA4) Name 2021	Text	ASGS Statistical Geography Standard
National LGAs Name 2024	Text	ASGS Statistical Geography Standard
National Road Type	Text	From Geoscape Australia
Christmas Period	Text	12 days from December 23
Easter Period	Text	5 days from Thursday before Good Friday
Age Group	Text	Standard road safety bulletin groupings
Day of week	Text	Weekday vs Weekend definition
Time of day	Text	Day (6am–5:59pm) vs Night

2 Design and Implementation

2.1 Design

Prior to creating the graph database in the Neo4j, the property graph was designed using Arrows App. With each node , its properties and relationships between them labelled. The property graph is given below (1)

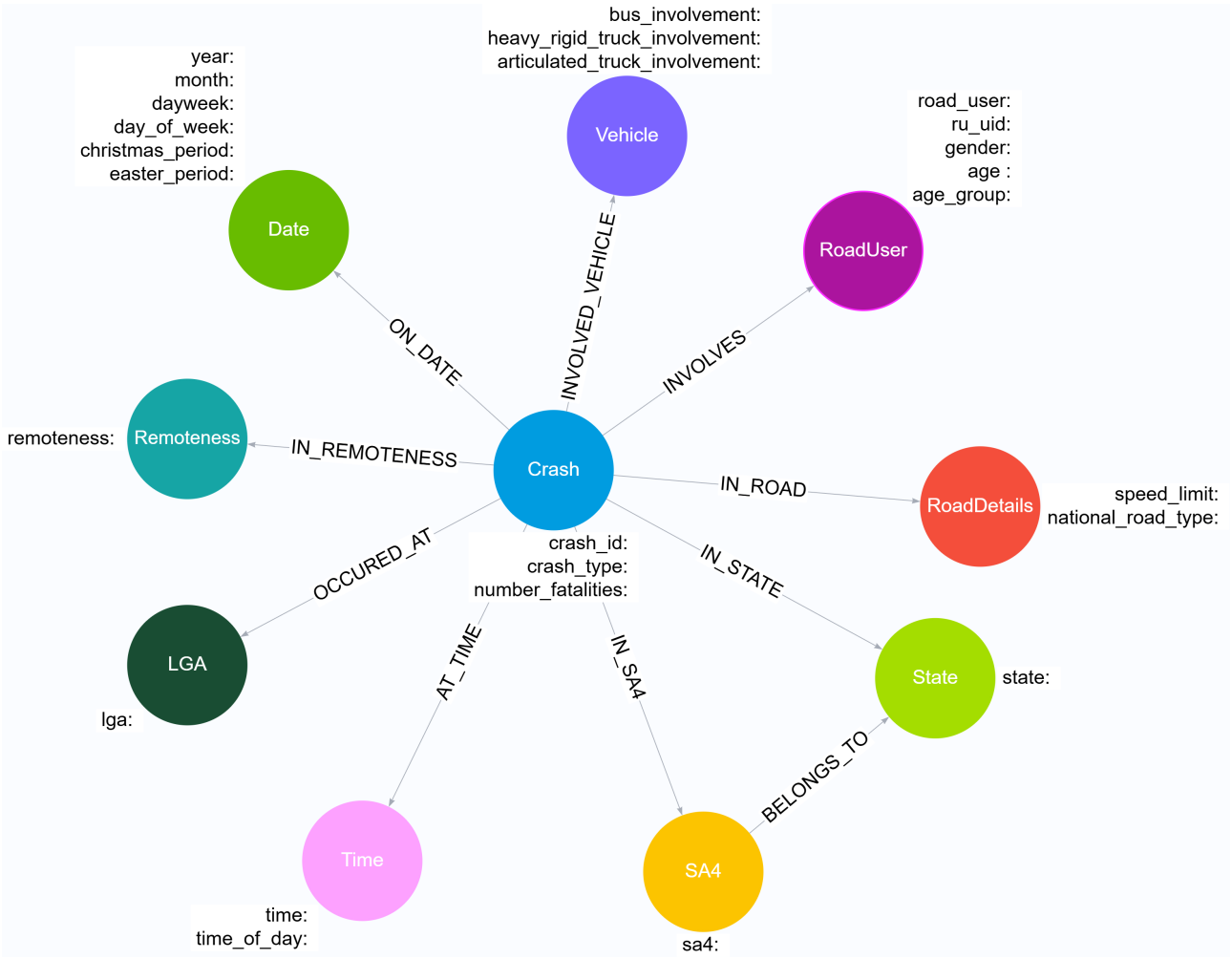


Figure 1: Property graph

In this property graph consists of the following nodes and relationships:

Nodes

- **Crash**: Central node representing each crash event. Contains attributes such as `crash_id`, `crash_type`, and `number_fatalities`.
- **RoadUser**: Represents individuals involved in the crash. Attributes include `road_user`, `ru_uid`, `gender`, `age`, and `age_group`.
- **Vehicle**: Represents the type of vehicle(s) involved in the crash. It includes properties indicating whether a `bus`, `heavy_rigid_truck`, or `articulated_truck` was involved (`bus_involvement`, `heavy_rigid_truck_involvement`, `articulated_truck_involvement`).
- **Date**: Represents the date of the crash. Includes `year`, `month`, `dayweek`, `day_of_week`, `christmas_period`, and

`easter_period`.

- **Time**: Describes the time of the crash, with properties such as `time` and `time_of_day`.
- **State**: Indicates the state in which the crash occurred.
- **LGA (Local Government Area)**: Specifies the local area where the crash took place.
- **SA4 (Statistical Area Level 4)**: Represents regional groupings used in Australian statistical geography.
- **Remoteness**: Refers to the ABS remoteness classification of the crash location.
- **RoadDetails**: Describes road characteristics including `speed_limit` and `national_road_type`.

Relationships

- **INVOLVES**: Each `Crash` is connected to one or more `RoadUser` nodes to show who was involved in the crash.
- **INVOLVED_VEHICLE**: Each `Crash` is connected to a `Vehicle` node to show which types of vehicles were involved.
- **ON_DATE**: Each `Crash` is connected to a `Date` node to record when the crash occurred.
- **AT_TIME**: Each `Crash` is connected to a `Time` node to show the time of day it happened.
- **IN_STATE**: Each `Crash` is connected to a `State` node to show which state it occurred in.
- **OCCURRED_AT**: Each `Crash` is connected to a `LGA` node to show the local area where it happened.
- **IN_SA4**: Each `Crash` is connected to an `SA4` node to show the broader region it belongs to.
- **BELONGS_TO**: Each `SA4` node is connected to a `State` node to show which state the region belongs to.
- **IN_REMOTENESS**: Each `Crash` is connected to a `Remoteness` node to show how urban or remote the crash location is.
- **IN_ROAD**: Each `Crash` is connected to a `RoadDetails` node to include road conditions like speed limit and road type.

2.2 Rationale Behind the Design Choices

The design behind the graph database for the fatal crash dataset was based on the principles of property graph modeling as discussed in the lecture 7[3]

The adopted model have:

- Nodes which represent Entities(for example Crash, RoadUser, Time) at that event

- Relationships represented the meaningful association between these entities(for example: which sa4 does the crash take place, which state does this specific sa4 belongs to etc)

- The properties store relevent information (for example: the age and gender of the RoadUser)

This aligns with the modeling practice where *nodes represent nouns, relationships represent verbs and the properties give context*[3]

2.2.1 Schema choice

The Design was chosen such that this graph database was to center the schema around the Crash node, with all other entities connected to it through relationships. This approach was chosen because:

- It makes sense in real world : The crashes are events that involve multiple entities, In the case of the traffic incidents , all the associated data(like who was killed, how many dies, where it occured , when it happened etc) all revolve around that single crash. So I have anchored the model around Crash, I have tried to keep the natural semantics of the data.

- For Efficient querying : most of the analysis tasks, like finding where the crash have happened more, at what time the crashes occur most etc, it all starts by identifying the crashes and then exploring the related data. Designing the graph with Crash in the center makes the traversal paths short and efficient, which is one of the desicions to make while designing the graph.

- Its modular and scalable : Since the Crash is used as the central node, Any additional node that may be considered for tracking (for example: the weather on the day, traffic control system nearby, alcohol or drug involvement, sleep deprivation or exhaustion etc) could be integrated seamlessly in the future to this graph by just simply adding another relationship from the Crash

- It is whiteboard friendly : As mentioned in the lecture, the graph modelling is done such that it could be drawn on a white board and make sense. This model is created in such way with the crash in center and everything else connected to it.

2.2.2 Some design justifications

- Since some road users may share the same role, age and gender in a crash, a unique identifier (`ru_uid`) was constructed which might help avoiding accidental merging of the deaths of seperate individuals.

- The location nodes are kept seperate seperate nodes which will support targeted queries by users only looking for one specific region/location This is much simpler than filtering nested location fields in a single node.

- Eventhough I have kept them in seperate nodes, the hierarchical relationship between the SA4 regions and the State have been modelled into the design, this will ensure that the geographic structure is retained still.

- Vehicle involvements (like bus involvement, heavy rigid truck involvement, articulated truck involvement) are aggregated under one node instead of multiple seperate nodes, which will simplify the query logic but still with enough expressiveness.

- The Date and time have been modelled seperarely, but i have kept all the holiday details(christmas day, easter day) all combined with the date, since they are inherently connected to dates.

3 Implementation

The graph was implemented using Neo4j .

- The dataset was preprocessed uing Python and Exported to CSV files for loading.

- Nodes and Relationships were created using Cypher Scripts via `LOAD CSV` command.

The process will be explained in detail in the sections below. While only representative code snippets and sample queries are included in the main report, the complete python code and Cypher Code willl be provided in the Appendix and will be submitted seperately along with the report

4 Creating CSV files from Dataset provided

This process was done in Python using VSCode. The dataset which was provided for the project was downloaded from the link given in the project specification[4], it was loaded into a dataframe. This process required to create the csv files which will be used for generating the nodes and relationships in the graph database.

4.1 Main dataset

After the dataset was loaded, basic cleaning was done, it could be seen in the figure(3) below

- Removed spaces and converted to lowercase in the column names

- Some of the column names(for LGA,remoteness,SA4) were renamed for ease of use and clarity

```
#basic dataset cleaning
og_data.columns = og_data.columns.str.strip().str.lower().str.replace(' ', '_')

og_data = og_data.rename(

    columns={"national_remoteness_areas": "remoteness",
             "sa4_name_2021": "sa4",
             "national_lga_name_2024": "lga"}
    )

og_data.head(3)
```

✓ 0.0s Python

	id	crash_id	state	month	year	dayweek	time	crash_type	number_fatalities	bus_involvement	...	age	remoteness
0	1	20241115	NSW	12	2024	Friday	4:00	Single	1	No	...	74	Inner Regional Australia
1	2	20241125	NSW	12	2024	Friday	6:15	Single	1	No	...	19	Inner Regional Australia
													Inner

Figure 2: Load dataset and clean headers

In the main dataset, when case of the **Crash and Road user** combinations, for example, if there were 2 pedestrians who died in same crash were twins(they have same age and gender), or if there was a bus accident and two individual passengers who are same gender and age died in the crash, those should be considered as two distinct instances, so each of these instances needs to be kept, therefore I chose to make an extra column(like a key) so the instances would stay distinct when counting.

```
og_data["road_user_id"] = og_data.groupby(["crash_id", "road_user", "gender", "age"]).cumcount()
#create additional property(like key) so that each combination of the crash_id and road_user is preserved
og_data["ru_uid"] = og_data["crash_id"].astype(str) + '_'
+ og_data["road_user"] + '_' + og_data["gender"] + '_'
+ og_data["age"].astype(str) + '_' + og_data["road_user_id"].astype(str)
```

Figure 3: extra column created in the dataset to preserve each distinct case of fatality

4.2 Creating csv files for nodes

While Creating the csv files for the nodes:

- The required columns were chosen from the provided dataset

- The duplicates were removed from these to reduce redundancy

- These were then saved to a csv file with index = false

Sample codes for the node creation are given below(4)

```
#creating nodes
og_data[["crash_id","crash_type","number_fatalities"]].drop_duplicates().to_csv("crash.csv",index=False)

og_data[["ru_uid","road_user","gender","age","age_group"]]\
    .drop_duplicates().to_csv("road_user.csv",index=False)
```

Figure 4: Sample code for node creation

4.3 Creating csv files for relationships

while creating the relationships, generally these steps are taken

- The required columns for the relationship were chosen from the original dataset.

- The duplicates were removed or kept based on the relationship(4.3.1)

- These were then saved to a csv file with index = false

4.3.1 Dealing with duplicates - Discussion

While creating the relationships, It was observed that each of the crash might have multiple rows for some relationships in the dataset if the same crash have been multiple fatalities associated with it. The duplicates from these relationships were removed, as the values are duplicated just because of the fatalities, so keeping the duplicates would be redundant.

So the **duplicates are removed** from these relationships :

- Crash and Road details

- Crash and Date

- Crash and Time

- Crash and vehicles involved

- Crash and State

- Crash and remoteness

- Crash and SA4

- Crash and LGA

In the relationship between **State and SA4**, also, the **duplicates were dropped** while creating the file as each SA4 belongs to a specific state, no matter how many crashes happen there.

But in case of the **Crash and Road user** relationship, since distinct instances were already taken care of using the distinct instances using the ”ru_uid”, In this case also i was chosen to **remove duplicates**

Sample code for creating some relationships are given below(5):

```python
#creating relationships
#crash - road user
og_data[["crash_id","ru_uid"]].drop_duplicates().to_csv("crash_user.csv",index=False)

#state - sa4
og_data[["sa4", "state"]].drop_duplicates().to_csv("sa4_state.csv", index=False)

#rest of them there would be multiple for the same crash if there has been multiple fatalities,
#so we should remove the duplicates to remove redundancy
og_data[["crash_id","time"]].drop_duplicates().to_csv("crash_time.csv",index=False)
og_data[["crash_id","state"]].drop_duplicates().to_csv("crash_state.csv",index=False)
```

Figure 5: sample ETL for relationships

5 Graph database Implementation

To implement the graph database in Neo4j, a new project was initialized within the Neo4j Desktop interface. The cleaned and preprocessed CSV files generated from the ETL process were moved into the designated 'import' directory of the Neo4j database. After launching the database instance, Cypher queries were executed through the Neo4j browser to load the data.

5.1 Loading Nodes

Sample for loading a node from the csv is given below(6)

```
LOAD CSV WITH HEADERS FROM 'file:///crash.csv' AS row
MERGE (c:Crash{crash_id : row.crash_id})
SET c.crash_type = row.crash_type,
c.number_fatalities = toInteger(row.number_fatalities);
```

Added 9683 labels, created 9683 nodes, set 29049 properties, completed after 7949 ms.

Figure 6: loading crash node from the csv

In this example, the query reads data from `crash.csv`, creates a `Crash` node for each row, and assigns it properties like `crash_type` and `number_fatalities`. As shown in the Neo4j browser output, 9683 crash nodes were successfully created.

5.2 Loading Relationships

Relationships were created between nodes to reflect real-world connections. For instance, the relationship `AT_TIME` connects a crash to the specific time it occurred, using values from `crash_time.csv`. An example query is shown below:(7)

```
1 LOAD CSV WITH HEADERS FROM 'file:///crash_time.csv' AS row
2 MATCH (c:Crash{crash_id:row.crash_id})
3 MATCH (t:Time {
4   time: row.time,
5   time_of_day: row.time_of_day
6 })
7 MERGE (c)-[:AT_TIME]→(t);
```

Table

Created 9683 relationships, completed after 22357 ms.

Figure 7: creating AT_TIME relationship from the csv

This query matches each crash to a corresponding `Time` node using the `time` and `time_of_day` fields, then creates an `AT_TIME` relationship between them. The result shows 9683 relationships were created, confirming that each crash was successfully connected to its associated time record.

This process was repeated to load all other entities and their relationships, such as date, location, road details, and road users. Once all data was loaded, the Neo4j browser could be used to visually inspect the graph and run Cypher queries for analysis.

After Loading the noodes and relationships, the database details can be seen from the Neo4j browser like this8

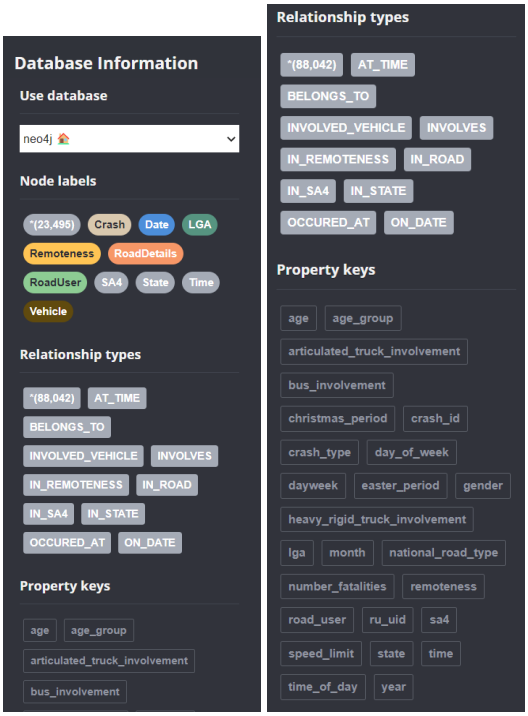


Figure 8: Database information from neo4j after loading the nodes and relationships in Neo4j

6 Cypher Query

Query a

Find all crashes in **WA** from **2020-2024** where **articulated trucks** were involved and multiple fatalities (**Number Fatalities > 1**) occurred. For each crash, provide the **road user**, **age** of each road user, **gender** of each road user, **LGA Name**, **month** and **year** of the crash, and the **total number of fatalities**.

```
1  MATCH (c:Crash)-[:INVOLVED_VEHICLE]→ (v:Vehicle),
2  (c)-[:IN_STATE]→ (s:State),
3  (c)-[:INVOLVES]→ (ru:RoadUser),
4  (c)-[:OCCURED_AT]→ (l:LGA),
5  (c)-[:ON_DATE]→ (d:Date)
6  WHERE s.state = "WA"
7  AND v.articulated_truck_involvement = "Yes"
8  AND c.number_fatalities > 1
9  AND d.year ⩾ 2020 AND d.year ⩽ 2024
10 RETURN ru.road_user as Road_User, ru.age as Age, ru.gender as Gender, l.lga AS LGA_Name,
   d.month as Month, d.year as Year, c.number_fatalities as Number_Of_Fatalities
```

Table | Text | Code

	Road_User	Age	Gender	LGA_Name	Month	Year	Number_Of_Fatalities
1	"Driver"	56	"Male"	"Dundas"	12	2020	2
2	"Driver"	58	"Male"	"Dundas"	12	2020	2
3	"Driver"	58	"Female"	"Busselton"	11	2020	2
4	"Passenger"	51	"Female"	"Busselton"	11	2020	2

Figure 9: screenshot of Neo4j for query a

- This query shows fatal crashes in Western Australia from 2020 to 2024 where articulated trucks were involved and more than one person died. It includes the road user type, their age and gender, the LGA name, and the crash date.
- The data shows two crashes: one in Dundas (Dec 2020) with two male drivers killed, and another in Busselton (Nov 2020) involving a female driver and passenger. Each crash had two fatalities.
- This suggests that regional areas with truck routes, like Dundas and Busselton, may have a higher risk of severe crashes. Safety improvements, truck regulations, or better signage could help reduce these fatal incidents.

Query b

Find the maximum and minimum age for female and male **motorcycle riders** who were involved in fatal crashes during the **Christmas Period** or **Easter Period** in **inner regional Australia**. Output the following information: **gender**, **maximum age** and **minimum age**.

```
MATCH (c:Crash)-[:INVOLVES]→(ru:RoadUser),
(c)-[:IN_REMOTENESS]→(r:Remoteness),
(c)-[:ON_DATE]→(d:Date)
WHERE toLower(ru.road_user) = "motorcycle rider"
AND r.remoteness = "Inner Regional Australia"
AND (d.christmas_period ="Yes" OR d.easter_period ="Yes")
AND c.number_fatalities> 0
RETURN ru.gender as Gender,min(ru.age) AS Minimum_age,max(ru.age) AS Maximum_age
```

Table | Text | Code

	Gender	Minimum_age	Maximum_age
1	"Male"	14	73

Figure 10: screenshot of Neo4j for query b

- This query looks at fatal crashes involving motorcycle riders during the Christmas or Easter period in inner regional Australia. It reports the minimum and maximum ages for each gender.
- The result shows that **male** motorcycle riders involved in fatal crashes ranged from **14 to 73 years old**. No data was returned for female riders.
- This may mean that no female motorcycle riders were fatally involved in such crashes during these periods and locations. It could reflect lower female ridership, or simply no recorded incidents in the data. Still, the wide age range among male riders highlights the need for holiday road safety messaging targeting both young and senior male motorcyclists.

Query c

How many young **drivers (Age Group = '17_to_25')** were involved in fatal crashes on weekends vs. weekdays in each state during **2024**? Output 4 columns: **State name, weekends, weekdays**, and the **average age for all young drivers (Age Group = '17_to_25') who were involved in fatal crashes in each State.**

```
MATCH (c:Crash)-[:INVOLVES]→(ru:RoadUser),
(c)-[:ON_DATE]→(d:Date),
(c)-[:IN_STATE]→(s:State)
WHERE ru.age_group = '17_to_25'
AND d.year = 2024
WITH s.state AS state,
d.day_of_week AS day,
ru.age AS age
RETURN state as State,
count(CASE WHEN day = 'Weekend' THEN 1 END) AS Weekends,
count(CASE WHEN day = 'Weekday' THEN 1 END) AS Weekdays,
round(avg(age), 3) AS Average_Age
```

Figure 11: Cypher query c

	State	Weekends	Weekdays	Average_Age
1	"NSW"	35	38	20.671
2	"TAS"	3	3	20.833
3	"QLD"	16	34	20.84
4	"VIC"	17	24	20.951
5	"SA"	3	9	20.417
6	"NT"	0	1	20.0
7	"ACT"	0	1	19.0

Figure 12: Answer c

- This query checks how many young drivers (age group `17_to_25`) were involved in fatal crashes on weekends versus weekdays in each Australian state during 2024. It also calculates the average age of those drivers in each state.
- The data shows that New South Wales (NSW) had the highest total number of fatal crashes involving young drivers, with 35 on weekends and 38 on weekdays. Other states like Queensland (QLD) and Victoria (VIC) also had significant counts, while Northern Territory (NT) and ACT had very few.
- This tells us that fatal crashes among young drivers happen on both weekdays and weekends across all states, with an average age close to 21. These results can help inform state-specific road safety programs that run throughout the week, not just on weekends.

Query d

Identify all crashes in **WA** that occurred **Friday** (but categorised as a **weekend**) and resulted in **multiple deaths**, with victims **being both male and female**. For each crash, output the **SA4 name**, **national remoteness areas**, and **national road type**.

```
MATCH (c:Crash)-[:ON_DATE]→(d:Date),
(c)-[:IN_STATE]→(s:State),
(c)-[:INVOLVES]→(ru:RoadUser),
(c)-[:IN_SA4]→(sa4:SA4),
(c)-[:IN_REMOTENESS]→(r:Remoteness),
(c)-[:IN_ROAD]→(rd:RoadDetails)
WHERE s.state = 'WA' AND d.dayweek = 'Friday' AND d.day_of_week = 'Weekend'
AND c.number_fatalities > 1
WITH c.crash_id AS Crash_ID, sa4.sa4 AS SA4, r.remoteness AS Remoteness_Area, rd.national_road_type AS Road_Type,
collect(DISTINCT ru.gender) AS genders
WHERE 'Male' IN genders AND 'Female' IN genders
RETURN Crash_ID, SA4, Remoteness_Area, Road_Type
```

Figure 13: Cypher query d

	Crash_ID	SA4	Remoteness_Area	Road_Type
1	"20155048"	"Western Australia - Outback (North)"	"Very Remote Australia"	"National or State Highway"
2	"20195098"	"Perth - South East"	"Major Cities of Australia"	"Local Road"

Figure 14: Answer d

- This query finds crashes in Western Australia that happened on a Friday, were considered part of the weekend, involved more than one fatality, and had both male and female victims.
- The data shows two such crashes: one in **Very Remote Australia** on a **National or State Highway**, and another in **Perth - South East** on a **Local Road**. These occurred in different SA4 regions with very different remoteness levels.
- This suggests that fatal crashes with mixed-gender victims and multiple deaths can occur both in remote regions and urban areas. It highlights the need for road safety strategies that are tailored to different geographic and road environments.

Query e

Find the top 5 SA4 regions where the highest number of fatal crashes occur during peak hours (**Time between 07:00-09:00 and 16:00-18:00**). For each SA4 region, output the **name** of the region and the separate number of crashes that occurred during morning peak hours and afternoon peak hours (Renamed Morning Peak and Afternoon Peak)

```
MATCH (c:Crash)-[:AT_TIME]→(t:Time),
(c)-[:IN_SA4]→(s:SA4)
WITH s.sa4 AS region,
toInteger(split(t.time, ":")[0]) AS hour
WITH region,
count(CASE WHEN hour ⩾ 7 AND hour ⩽ 9 THEN 1 END) AS Morning_Peak,
count(CASE WHEN hour ⩾ 16 AND hour ⩽ 18 THEN 1 END) AS Afternoon_Peak
RETURN region, Morning_Peak, Afternoon_Peak
ORDER BY (Morning_Peak + Afternoon_Peak) DESC
LIMIT 5
```

Figure 15: Cypher query e

	region	Morning_Peak	Afternoon_Peak
1	"South Australia - South East"	36	49
2	"Wide Bay"	31	47
3	"Melbourne - South East"	30	42
4	"Capital Region"	34	37
5	"New England and North West"	24	41

Figure 16: Answer e

- This query finds the top 5 SA4 regions with the highest number of fatal crashes that happened during peak traffic hours — specifically between 07:00–09:00 (morning peak) and 16:00–18:00 (afternoon peak).
- The results show that **South Australia - South East** and **Wide Bay** have the highest fatal crash counts during these hours. All top 5 regions show more crashes during the afternoon peak than the morning.
- This suggests that afternoon peak hours may be more dangerous than mornings in many regions. Contributing factors could include end-of-day fatigue, school traffic, or heavier congestion. Authorities could consider peak-hour patrols, better signage, or awareness campaigns targeted at afternoon commuters.

Query f

Find paths with a length of 3 between any two LGAs. **Return the top 3 paths, including the starting LGA and ending LGA for each path.** Order results alphabetically by starting LGA and then ending LGA.

```
MATCH path = (start:LGA)-[*3]-(end:LGA)
WHERE start <> end
WITH start, end, path
ORDER BY start.lga ASC, end.lga ASC
RETURN start.lga AS Start_LGA, end.lga AS End_LGA, path
LIMIT 3
```

(no changes, no records)

Figure 17: screenshot of Neo4j for query f

- This query tries to find paths of exactly 3 relationships between two different LGAs. It returns the top 3 such paths, ordered alphabetically by starting and ending LGA names.
- However, no results were returned. This is because of the graph design: the schema is centered around the `Crash` node, and all other nodes (including LGAs) connect to it directly. LGAs do not have direct or indirect relationships with each other in the current setup.
- Since LGAs are not connected to one another through a shared entity like a road network or region node, it is not possible to traverse from one LGA to another within 3 hops in this database design.

Query g

Find all weekday fatal crashes involving **pedestrians** where either **buses or heavy rigid trucks** were present in **speed zones less than 40 or greater than/equal to 100**. Group these crashes by unique combinations of **time of day, age group, vehicle type** (bus or heavy rigid truck), and **speed limitation**. For each group, count the number of crashes that occurred. Output a table showing **time of day, age group, vehicle type, crash count**, and **speed limitation**, sorted first by **time of day** (ascending) and then by **age group** (ascending)

```
MATCH (c:Crash)-[:INVOLVES]→(ru:RoadUser),(c)-[:ON_DATE]→(d:Date),
(c)-[:AT_TIME]→(t:Time),(c)-[:IN_ROAD]→(rd:RoadDetails),
(c)-[:INVOLVED_VEHICLE]→(v:Vehicle)
WHERE ru.road_user = "Pedestrian" AND d.day_of_week = "Weekday" AND c.number_fatalities > 0
AND (v.bus_involvement ="Yes" OR v.heavy_rigid_truck_involvement ="Yes") AND (rd.speed_limit < 40 OR rd.speed_limit ⩾ 100)
WITH t.time_of_day AS time_of_day,ru.age_group AS age_group,
  CASE
    WHEN v.bus_involvement = "Yes" THEN "Bus"
    ELSE 'Heavy Rigid Truck'
  END AS vehicle_type,rd.speed_limit AS speed_limit,count(DISTINCT c.crash_id) AS Crash_Count
RETURN time_of_day AS Time_Of_Day, age_group AS Age_Group, vehicle_type AS Vehicle_Type, speed_limit AS Speed_Limit, Crash_Count
ORDER BY Time_Of_Day ASC, Age_Group ASC
```

Figure 18: Cypher query g

	Time_Of_Day	Age_Group	Vehicle_Type	Speed_Limit	Crash_Count
1	"Day"	"0_to_16"	"Heavy Rigid Truck"	110	1
2	"Day"	"17_to_25"	"Heavy Rigid Truck"	20	1
3	"Day"	"26_to_39"	"Heavy Rigid Truck"	100	2
4	"Day"	"40_to_64"	"Heavy Rigid Truck"	100	1
5	"Day"	"40_to_64"	"Heavy Rigid Truck"	110	2
6	"Day"	"40_to_64"	"Bus"	10	1
7					

rted streaming 11 records after 30 ms and completed after 33 ms.

Figure 19: Answer g

- This query looks at fatal weekday crashes involving pedestrians where either a bus or a heavy rigid truck was involved, and the crash happened in a speed zone less than 40 or at least 100 km/h. It groups the results by time of day, age group, vehicle type, and speed limit.
- The data shows that **most crashes happened during the day**, and almost all involved **heavy rigid trucks**. Pedestrians from a wide range of age groups, including children and seniors, were affected. A few crashes also involved buses at very low speeds (10–20 km/h).
- This suggests that both very slow and very fast speed zones can be dangerous for pedestrians when large vehicles are involved. Daytime crashes may reflect school or work commute risks. These results support the need for better pedestrian safety measures, like crossings and vehicle speed enforcement, especially during the day.

7 Self Designed Queries

Monthly trend for fatality for young people

```
MATCH (c:Crash)-[:INVOLVES]→(ru:RoadUser),
      (c)-[:ON_DATE]→(d:Date)
WHERE ru.age_group = '17_to_25'
RETURN d.month AS Month, count(*) AS Crash_Count
ORDER BY Crash_Count DESC
```

Table | Text | Code

	Month	Crash_Count
1	12	203
2	9	191
3	3	185
4	4	183
5	1	174
6	7	168

Figure 20: screenshot of Neo4j for monthly trend for fatality for young people

What it shows: This query tells us which months had the most crashes involving young drivers.

Why it matters: Young drivers are often less experienced. Knowing when they are most at risk helps plan road safety campaigns and improve traffic rules during those months.

What the data tells: If crash numbers are high in months like December or January, it might be due to holidays, school breaks, or parties. These months could be targeted with safety messages or temporary traffic changes.

Crash Frequency by Road Type and Time of Day

```
MATCH (c:Crash)-[:IN_ROAD]→(rd:RoadDetails),
      (c)-[:AT_TIME]→(t:Time)
RETURN rd.national_road_type AS Road_Type,
       t.time_of_day AS Time_Of_Day,
       count(*) AS Crash_Count
ORDER BY Crash_Count DESC
```

	Road_Type	Time_Of_Day	Crash_Count
1	"National or State Highway"	"Day"	1677
2	"Arterial Road"	"Day"	1393
3	"Local Road"	"Day"	1188
4	"National or State Highway"	"Night"	1077
5	"Sub-arterial Road"	"Day"	1007
6	"Arterial Road"	"Night"	890

Figure 21: screenshot of Neo4j for crash frequency by road type and time of day

What it shows: This shows which road types and times of day have the highest number of crashes.

Why it matters: Some roads are more dangerous at certain times. This information helps in deciding where to improve lighting, put up warning signs, or increase patrols.

What the data tells: If most crashes happen on highways in the evening, drivers might be tired or visibility may be low. Fixes could include better lighting or adjusting speed limits.

LGAs with Frequent Crashes

```
MATCH (l:LGA)←[:OCCURED_AT]-(c:Crash)
WITH l.lga AS LGA, count(*) AS Crash_Count
WHERE Crash_Count > 5
RETURN LGA, Crash_Count
ORDER BY Crash_Count DESC
LIMIT 10
```

	LGA	Crash_Count
1	"Brisbane"	194
2	"Gold Coast"	125
3	"Moreton Bay"	125
4	"Central Coast (NSW)"	114
5	"Sunshine Coast"	107
6	"Logan"	103
7		

Figure 22: screenshot of Neo4j for LGAs with Frequent Crashes

What it shows: This query finds the local areas (LGAs) that had the most crashes.

Why it matters: It highlights crash hotspots. These areas may need urgent attention for road upgrades or more traffic control.

What the data tells: If a place like “City of Wanneroo” has many crashes, it might be due to heavy traffic or bad intersections. Councils can use this to plan improvements or request government funding.

Fatal Crashes by Age Group and Time of Day

```
MATCH (c:Crash)-[:INVOLVES]→(ru:RoadUser),
      (c)-[:AT_TIME]→(t:Time)
WHERE c.number_fatalities > 0
RETURN ru.age_group AS Age_Group, t.time_of_day AS Time_Of_Day, count(*) AS Crash_Count
ORDER BY Crash_Count DESC
```

	Age_Group	Time_Of_Day	Crash_Count
1	"40_to_64"	"Day"	2054
2	"26_to_39"	"Night"	1201
3	"40_to_64"	"Night"	1194
4	"17_to_25"	"Night"	1144
5	"26_to_39"	"Day"	1112

Figure 23: screenshot of Neo4j for fatal crashes by age group and time of day

What it shows: This shows which age groups are dying in crashes and during what time of the day.

Why it matters: Different age groups face different risks. Knowing when they are most at risk helps tailor safety rules and messages.

What the data tells: Most people aged 40–64 are dying in daytime crashes—possibly because of work travel. Young people (17–25) are dying more at night, likely due to partying, fatigue, or speeding. Authorities can use this to plan night driving restrictions or safety ads for young drivers.

8 Applications of Graph Data Science

Graph Dara Science gives a powerful framework for solving problems with some of the most complex relaionships in them. one of the most widely used graph algorithm in the world today is PageRank, which is a link analysis algorithm which was originally designed by google to rank websites based on their importance. This algorithm have been adapted to solve a wide range of real-world problems beyond search engines . This is also actively used across various industries.

The algorithm PageRank operates by assigning a score to each of the nodes in the graph based on the importance of other nodes linking to it. It uses the idea that if a node is connected to other important nodes, then it is important. This method works well for directed graphs, such as web pages, transaction networks, or citation networks.

Real-World Applications of PageRank

PageRank is actively used today in the following domains:

- **Search engines:** Google's early success was built on PageRank. Although modern search uses hundreds of signals, PageRank remains one of the foundational components for measuring link-based importance [5].
- **Fraud detection and finance:** Companies like PayPal and firms using Neo4j use PageRank to detect money

laundering schemes. Accounts receiving funds from many influential or suspicious sources are flagged based on their centrality in the transaction network [6].

- **Cybersecurity:** In the MITRE ATT&CK framework, PageRank is used to prioritize nodes in cyberattack graphs by importance, helping security analysts focus on the most critical assets and potential attack paths [7].
- **Healthcare and biology:** In gene-disease association studies, tools like GeneRank use PageRank to prioritize genes in interaction networks, ranking those most likely to be disease-relevant [8].
- **Recommendation systems:** Platforms like eBay use PageRank on user-item interaction graphs to rank sellers, items, or user trust levels [6].

References

[1] "Cits5504 project 2 - graph database design and cypher query, corrected dataset," https://618598184-files.gitbook.io/~/files/v0/b/gitbook-x-prod.appspot.com/o/spaces%2FDoBTcHHBV80kLIWxmuWv%2Fuploads%2FtENZqqLI6Gq8KHuaRbvl%2FProject2_Dataset_Corrected.csv?alt=media&token=24ceee2f-7372-4b7f-a78e-d075ec59dea0, Department of Computer Science and Software Engineering, School of Physics, Mathematics and Computing, University of Western Australia, 2025, section 3, Perth, WA, Australia.

[2] Bureau of Infrastructure and Transport Research Economics, "Road deaths australia—tables—december 2024," https://www.bitre.gov.au/sites/default/files/documents/bitre_fatalities_dec2024.xlsx, 2024, accessed: May 22, 2025.

[3] J. Li, "Lecture 7 – intro to graph databases and data modelling: From relational to graphs," Lecture slides, 2025, department of Computer Science and Software Engineering, School of Physics, Mathematics and Computing, University of Western Australia, Perth, WA, Australia.

[4] Department of Computer Science and Software Engineering, "Project 2 - graph database design and cypher query," https://csse-uwa.gitbook.io/data-warehouse-project-2-s1-2025, 2025, school of Physics, Mathematics and Computing, University of Western Australia, Perth, WA, Australia. Accessed: May 24, 2025.

[5] S. Brin and L. Page, "The anatomy of a large-scale hypertextual web search engine," *Computer Networks and ISDN Systems*, vol. 30, no. 1–7, pp. 107–117, 1998.

[6] Neo4j Inc., "Pagerank algorithm - graph data science," https://neo4j.com/docs/graph-data-science/current/algorithms/page-rank/, 2024, accessed: 2025-05-19.

[7] P. C. Johnson, C. L. Toma, M. Bishop, and D. G. Shackleford, "Using graph algorithms to prioritize cyber threats," in *MILCOM 2015 - 2015 IEEE Military Communications Conference*, 2015, pp. 1441–1446.

[8] A. Vandin, A. Bassi, G. Franchi, and M. Pellegrini, "Generank: A pagerank-based gene prioritization tool for disease relevance," *BMC Bioinformatics*, vol. 22, no. 1, pp. 1–12, 2021.

[9] OpenAI, "Chatgpt (gpt-4)," https://chat.openai.com, 2023, used for writing refinement and code debugging assistance.

A Appendix

Python Preprocessing Code

Listing 1: Libraries

```
#libraries
import subprocess
import sys

for pkg in ["pandas","requests"]:
    try:
        __import__(pkg)
    except ImportError:
        subprocess.check_call([sys.executable, "-m","pip","install",pkg])

import pandas as pd
import requests
```

Listing 2: download the dataset from the url

```
#download the dataset from the url
url = "https://618598184-files.gitbook.io/~/files/v0/b/gitbook-x-prod.appspot.com/o/
    spaces%2FDoBTcHHBV80kLIWxmuWv%2Fuploads%2FtENZqqLI6Gq8KHuaRbvl%2
    FProject2_Dataset_Corrected.csv?alt=media&token=24ceee2f-7372-4b7f-a78e-d075ec59dea0"
filename = "Project2_Dataset_Corrected.csv"

with open(filename,"wb") as f:
    f.write(requests.get(url).content)

#load the dataset
og_data = pd.read_csv(filename)
```

Listing 3: basic dataset cleaning and creating extea column

```
#basic dataset cleaning
og_data.columns = og_data.columns.str.strip().str.lower().str.replace(' ', '_')

og_data = og_data.rename(

    columns={"national_remoteness_areas": "remoteness",
             "sa4_name_2021": "sa4",
             "national_lga_name_2024": "lga"}
    )

#create additional column
og_data["road_user_id"] = og_data.groupby(["crash_id", "road_user", "gender", "age"]).
    cumcount()
#create additional property(like key) so that each combination of the crash_id and
    road_user is preserved
```

```
og_data["ru_uid"] = og_data["crash_id"].astype(str) + '_' + og_data["road_user"] + '_' +
    og_data["gender"] + '_' + og_data["age"].astype(str) + '_' + og_data["road_user_id"].
    astype(str)
```

Listing 4: creating nodes

```
#creating nodes

og_data[["crash_id","crash_type","number_fatalities"]].drop_duplicates().to_csv("crash.
    csv",index=False)

og_data[["ru_uid","road_user","gender","age","age_group"]]\
    .drop_duplicates().to_csv("road_user.csv",index=False)

og_data[["bus_involvement","heavy_rigid_truck_involvement","articulated_truck_involvement
    "]].drop_duplicates().to_csv("vehicles.csv",index=False)

og_data[["speed_limit","national_road_type"]].drop_duplicates().to_csv("road_details.csv"
    ,index=False)

og_data[["year","month","dayweek","day_of_week","christmas_period","easter_period"]].
    drop_duplicates().to_csv("date.csv",index=False)

og_data[["time","time_of_day"]].drop_duplicates().to_csv("time.csv",index=False)

og_data[["state"]].drop_duplicates().to_csv("state.csv",index=False)

og_data[["remoteness"]].drop_duplicates().to_csv("remoteness.csv",index=False)

og_data[["sa4"]].drop_duplicates().to_csv("sa4.csv",index=False)

og_data[["lga"]].drop_duplicates().to_csv("lga.csv",index=False)
```

Listing 5: creating relationships

```
#creating relationships
#crash - road user
og_data[["crash_id","ru_uid"]].drop_duplicates().to_csv("crash_user.csv",index=False)

#state - sa4
og_data[["sa4", "state"]].drop_duplicates().to_csv("sa4_state.csv", index=False)

#rest of them there would be multiple for the same crash if there has been multiple #
    fatalities, so we should remove the duplicates to remove redundancy
og_data[["crash_id","time","time_of_day"]].drop_duplicates().to_csv("crash_time.csv",
    index=False)
og_data[["crash_id","state"]].drop_duplicates().to_csv("crash_state.csv",index=False)

og_data[["crash_id","speed_limit","national_road_type"]].drop_duplicates().to_csv("
    crash_road.csv",index=False)

og_data[["crash_id","year","month","dayweek","day_of_week","christmas_period","
    easter_period"]].drop_duplicates().to_csv("crash_date.csv",index=False)
```

```
og_data[["crash_id","bus_involvement","heavy_rigid_truck_involvement","
    articulated_truck_involvement"]].drop_duplicates().to_csv("crash_vehicle.csv",index=
    False)

og_data[["crash_id","remoteness"]].drop_duplicates().to_csv("crash_remoteness.csv",index=
    False)

og_data[["crash_id","sa4"]].drop_duplicates().to_csv("crash_sa4.csv",index=False)

og_data[["crash_id","lga"]].drop_duplicates().to_csv("crash_lga.csv",index=False)
```

Cypher Script

Loading data into Neo4j

Listing 6: Loading Nodes

```
//Crash

LOAD CSV WITH HEADERS FROM 'file:///crash.csv' AS row
MERGE (c:Crash{crash_id : row.crash_id})
SET c.crash_type = row.crash_type,
c.number_fatalities = toInteger(row.number_fatalities);


//road user

LOAD CSV WITH HEADERS FROM 'file:///road_user.csv' AS row
MERGE (ru:RoadUser {ru_uid: row.ru_uid})
SET ru.road_user = row.road_user,
    ru.gender = row.gender,
    ru.age = toInteger(row.age),
    ru.age_group = row.age_group;



//vehicle

LOAD CSV WITH HEADERS FROM 'file:///vehicles.csv' AS row
MERGE(v:Vehicle {
bus_involvement: row.bus_involvement,
heavy_rigid_truck_involvement: row.heavy_rigid_truck_involvement,
articulated_truck_involvement: row.articulated_truck_involvement
});


//road details

LOAD CSV WITH HEADERS FROM 'file:///road_details.csv' AS row
MERGE (rd:RoadDetails{
speed_limit:toInteger(row.speed_limit),
national_road_type: row.national_road_type
});


//time

LOAD CSV WITH HEADERS FROM 'file:///time.csv' AS row
MERGE (t:Time{
time: row.time,
time_of_day: row.time_of_day
});
```

```
//date

LOAD CSV WITH HEADERS FROM 'file:///date.csv' AS row
MERGE (d:Date{
year:toInteger(row.year),
month:toInteger(row.month),
dayweek: row.dayweek,
day_of_week: row.day_of_week,
christmas_period: row.christmas_period,
easter_period: row.easter_period
});


//locations


//LGA

LOAD CSV WITH HEADERS FROM 'file:///lga.csv' AS row
MERGE (l:LGA{
lga: row.lga
});


//SA4

LOAD CSV WITH HEADERS FROM 'file:///sa4.csv' AS row
MERGE (sa:SA4{
sa4: row.sa4
});

//Remoteness

LOAD CSV WITH HEADERS FROM 'file:///remoteness.csv' AS row
MERGE (r:Remoteness{
remoteness: row.remoteness
});

//State

LOAD CSV WITH HEADERS FROM 'file:///state.csv' AS row
MERGE (s:State{
state: row.state
});
```

Listing 7: Loading Relationships

```
// Crash -> RoadUser
LOAD CSV WITH HEADERS FROM 'file:///crash_user.csv' AS row
MATCH (c:Crash{crash_id:row.crash_id})
MATCH (ru:RoadUser {ru_uid: row.ru_uid})
MERGE (c)-[:INVOLVES]->(ru);

// Crash -> RoadDetails
```

```
LOAD CSV WITH HEADERS FROM 'file:///crash_road.csv' AS row
MATCH (c:Crash{crash_id:row.crash_id})
MATCH (rd:RoadDetails{
speed_limit:toInteger(row.speed_limit),
national_road_type: row.national_road_type
})
MERGE (c)-[:IN_ROAD]->(rd);

// Crash -> Vehicle
LOAD CSV WITH HEADERS FROM 'file:///crash_vehicle.csv' AS row
MATCH (c:Crash{crash_id:row.crash_id})
MATCH (v:Vehicle {
bus_involvement: row.bus_involvement,
heavy_rigid_truck_involvement: row.heavy_rigid_truck_involvement,
articulated_truck_involvement: row.articulated_truck_involvement
})
MERGE (c)-[:INVOLVED_VEHICLE]->(v);


// Crash -> Date
LOAD CSV WITH HEADERS FROM 'file:///crash_date.csv' AS row
MATCH (c:Crash{crash_id:row.crash_id})
MATCH (d:Date{
year:toInteger(row.year),
month:toInteger(row.month),
dayweek: row.dayweek,
day_of_week: row.day_of_week,
christmas_period: row.christmas_period,
easter_period: row.easter_period
})
MERGE (c)-[:ON_DATE]->(d);

// Crash -> Remoteness
LOAD CSV WITH HEADERS FROM 'file:///crash_remoteness.csv' AS row
MATCH (c:Crash{crash_id:row.crash_id})
MATCH (r:Remoteness{
remoteness: row.remoteness
})
MERGE (c)-[:IN_REMOTENESS]->(r);

// Crash -> Time
LOAD CSV WITH HEADERS FROM 'file:///crash_time.csv' AS row
MATCH (c:Crash{crash_id:row.crash_id})
MATCH (t:Time{
time: row.time,
time_of_day: row.time_of_day
})
MERGE (c)-[:AT_TIME]->(t);

// Crash -> LGA
LOAD CSV WITH HEADERS FROM 'file:///crash_lga.csv' AS row
MATCH (c:Crash{crash_id:row.crash_id})
MATCH (l:LGA{
```

```
lga: row.lga
})
MERGE (c)-[:OCCURED_AT]->(l);

// Crash -> SA4
LOAD CSV WITH HEADERS FROM 'file:///crash_sa4.csv' AS row
MATCH (c:Crash{crash_id:row.crash_id})
MATCH (sa:SA4{
sa4: row.sa4
})
MERGE (c)-[:IN_SA4]->(sa);

// Crash -> State
LOAD CSV WITH HEADERS FROM 'file:///crash_state.csv' AS row
MATCH (c:Crash{crash_id:row.crash_id})
MATCH (s:State{
state: row.state
})
MERGE (c)-[:IN_STATE]->(s);

// SA4 -> State
LOAD CSV WITH HEADERS FROM 'file:///sa4_state.csv' AS row
MATCH (sa:SA4{
sa4: row.sa4
})
MATCH (s:State{
state: row.state
})
MERGE (sa)-[:BELONGS_TO]->(s);
```

Cypher Query for answering the Questions

Listing 8: Query for given Questions

```
//Query a
MATCH (c:Crash)-[:INVOLVED_VEHICLE]-> (v:Vehicle),
(c)-[:IN_STATE]-> (s:State),
(c)-[:INVOLVES]-> (ru:RoadUser),
(c)-[:OCCURED_AT]-> (l:LGA),
(c)-[:ON_DATE]-> (d:Date)
WHERE s.state = "WA"
AND v.articulated_truck_involvement = "Yes"
AND c.number_fatalities > 1
AND d.year >= 2020 AND d.year <= 2024
RETURN ru.road_user as Road_User, ru.age as Age, ru.gender as Gender, l.lga AS LGA_Name,
    d.month as Month, d.year as Year, c.number_fatalities as Number_Of_Fatalities



//Query b
MATCH (c:Crash)-[:INVOLVES]->(ru:RoadUser),
(c)-[:IN_REMOTENESS]->(r:Remoteness),
(c)-[:ON_DATE]->(d:Date)
```

```
WHERE toLower(ru.road_user) = "motorcycle rider"
AND r.remoteness = "Inner Regional Australia"
AND (d.christmas_period ="Yes" OR d.easter_period ="Yes")
AND c.number_fatalities> 0
RETURN ru.gender as Gender,min(ru.age) AS Minimum_age,max(ru.age) AS Maximum_age

//Query c
MATCH (c:Crash)-[:INVOLVES]->(ru:RoadUser),
(c)-[:ON_DATE]->(d:Date),
(c)-[:IN_STATE]->(s:State)
WHERE ru.age_group = '17_to_25'
AND d.year = 2024
WITH s.state AS state,
d.day_of_week AS day,
ru.age AS age
RETURN state as State,
count(CASE WHEN day = 'Weekend' THEN 1 END) AS Weekends,
count(CASE WHEN day = 'Weekday' THEN 1 END) AS Weekdays,
round(avg(age), 3) AS Average_Age

//Query d
MATCH (c:Crash)-[:ON_DATE]->(d:Date),
(c)-[:IN_STATE]->(s:State),
(c)-[:INVOLVES]->(ru:RoadUser),
(c)-[:IN_SA4]->(sa4:SA4),
(c)-[:IN_REMOTENESS]->(r:Remoteness),
(c)-[:IN_ROAD]->(rd:RoadDetails)
WHERE s.state = 'WA' AND d.dayweek = 'Friday' AND d.day_of_week = 'Weekend'
AND c.number_fatalities > 1
WITH c.crash_id AS Crash_ID, sa4.sa4 AS SA4, r.remoteness AS Remoteness_Area, rd.
    national_road_type AS Road_Type,
collect(DISTINCT ru.gender) AS genders
WHERE 'Male' IN genders AND 'Female' IN genders
RETURN Crash_ID, SA4, Remoteness_Area, Road_Type


//Query e
MATCH (c:Crash)-[:AT_TIME]->(t:Time),
(c)-[:IN_SA4]->(s:SA4)
WITH s.sa4 AS region,
toInteger(split(t.time, ":")[0]) AS hour
WITH region,
count(CASE WHEN hour >= 7 AND hour <= 9 THEN 1 END) AS Morning_Peak,
count(CASE WHEN hour >= 16 AND hour <= 18 THEN 1 END) AS Afternoon_Peak
RETURN region, Morning_Peak, Afternoon_Peak
ORDER BY (Morning_Peak + Afternoon_Peak) DESC
LIMIT 5


// Query f
MATCH path = (start:LGA)-[*3]-(end:LGA)
WHERE start <> end
WITH start, end, path
```

```
ORDER BY start.lga ASC, end.lga ASC
RETURN start.lga AS Start_LGA, end.lga AS End_LGA, path
LIMIT 3


//Query g
MATCH (c:Crash)-[:INVOLVES]->(ru:RoadUser),(c)-[:ON_DATE]->(d:Date),
(c)-[:AT_TIME]->(t:Time),(c)-[:IN_ROAD]->(rd:RoadDetails),
(c)-[:INVOLVED_VEHICLE]->(v:Vehicle)
WHERE ru.road_user = "Pedestrian" AND d.day_of_week = "Weekday" AND c.number_fatalities >
     0
AND (v.bus_involvement ="Yes" OR v.heavy_rigid_truck_involvement ="Yes") AND (rd.
    speed_limit < 40 OR rd.speed_limit >= 100)
WITH t.time_of_day AS time_of_day,ru.age_group AS age_group,
  CASE
    WHEN v.bus_involvement = "Yes" THEN "Bus"
    ELSE 'Heavy Rigid Truck'
  END AS vehicle_type,rd.speed_limit AS speed_limit,count(DISTINCT c.crash_id) AS
      Crash_Count
RETURN time_of_day AS Time_Of_Day, age_group AS Age_Group, vehicle_type AS Vehicle_Type,
    speed_limit AS Speed_Limit, Crash_Count
ORDER BY Time_Of_Day ASC, Age_Group ASC
```

Listing 9: Self designed Queries

```
// Self Queries

//Query 1
MATCH (c:Crash)-[:INVOLVES]->(ru:RoadUser),
      (c)-[:ON_DATE]->(d:Date)
WHERE ru.age_group = '17_to_25'
RETURN d.month AS Month, count(*) AS Crash_Count
ORDER BY Crash_Count DESC

//Query 2
MATCH (c:Crash)-[:IN_ROAD]->(rd:RoadDetails),
      (c)-[:AT_TIME]->(t:Time)
RETURN rd.national_road_type AS Road_Type,
       t.time_of_day AS Time_Of_Day,
       count(*) AS Crash_Count
ORDER BY Crash_Count DESC


//Query 3
MATCH (l:LGA)<-[:OCCURED_AT]-(c:Crash)
WITH l.lga AS LGA, count(*) AS Crash_Count
WHERE Crash_Count > 5
RETURN LGA, Crash_Count
ORDER BY Crash_Count DESC
LIMIT 10

//Query 4
```

```
MATCH (c:Crash)-[:INVOLVES]->(ru:RoadUser),
      (c)-[:AT_TIME]->(t:Time)
WHERE c.number_fatalities > 0
RETURN ru.age_group AS Age_Group, t.time_of_day AS Time_Of_Day, count(*) AS Crash_Count
ORDER BY Crash_Count DESC
```

Use of AI Tools

During the preparation of this report, OpenAI's ChatGPT (GPT-4)[9] was used to enhance the clarity, grammar, and structure of the writing. The tool was also consulted for assistance in resolving Python and Cypher code errors by providing suggestions and debugging support.